Assignment 1

Understanding LLMs, Prompting, and APIs

Mudil Goel
Roll No: 24B3932
Department of Electrical Engineering, IIT Bombay | June 2026

Question 1: Large Language Model Context Window Limitations

The Problem: Token Limits and Truncation

Large Language Models (LLMs) process text in units called "tokens," where one token is roughly equivalent to 0.75 words. Assuming an average of 160 words per page (8 words per line $\times$ 20 lines per page), a 50-page medical history document contains approximately 8,000 words. This translates to roughly $10,666$ tokens ($\approx 8,000 \times \frac{4}{3}$).

When this input is fed into an LLM with a 4,000-token context window, the model physically cannot process the entire document simultaneously. The system handles this overflow through **truncation**. Operating on a First-In, First-Out (FIFO) memory basis, the model will discard the earliest $\approx 6,666$ tokens, resulting in the complete loss of the beginning of the patient's medical history.

The Consequences: Hallucination in a Medical Context

Exceeding the context window drastically reduces the model's reliability. Because the LLM attempts to answer the user's prompt without access to the truncated sections of the text, it is highly prone to **hallucination**. In the context of a hospital chatbot, the model may confidently generate plausible but factually incorrect medical data, posing a severe safety risk to patient care.

Proposed Solutions

To mitigate this limitation, the following practical approaches can be implemented:

- **Utilize a Model with a Larger Context Window:** Upgrade to an LLM specifically designed for long-context tasks, allowing the entire document to be ingested at once without truncation.
- **Chunking and Summarization:** Instead of passing the entire document at once, divide the 50 pages into smaller "chunks" (e.g., 5 pages at a time). The LLM can be prompted to summarize each chunk individually, and those summaries can then be combined into a much shorter master document that easily fits within the 4,000-token limit.
- **Enforce User Specificity / Manual Retrieval:** Design the chatbot's interface to prompt the user for specific parameters (e.g., requesting a particular page number, date range, or medical section) so that only the relevant portion of the text is extracted and fed into the prompt.

Question 2: Study Buddy Bot using the Anthropic API

Script Overview

The following Python script utilizes the official Anthropic API to create a simple "Study Buddy" bot. It takes user input, performs basic validation to ensure the input is not empty, and sends a request to the `claude-haiku-4-5-20251001` model using the required Messages format. The response is then parsed and printed to the terminal.

Python Implementation

```python
import anthropic
import os

# --- 1. SETUP ---
API_KEY = ""

# Creating the client that will carry our messages to Claude
client = anthropic.Anthropic(api_key=API_KEY)

def main():
    print("Welcome to the Study Buddy Bot!")
    print("Type 'exit' to quit.\n")

    while True:
        # --- 2. GET USER INPUT ---
        # The script asks the user for a topic.
        topic = input("Enter a topic you want explained: ")

        # --- 3. INPUT VALIDATION ---
        # We check if the input is empty or just spaces.
        # .strip() removes whitespace from the beginning and end.
        if not topic.strip():
            print("Error: Please enter a valid topic. You cannot leave it blank!\n")
            continue  # Skips the rest of the loop and asks again

        if topic.lower() == 'exit':
            print("Goodbye!")
            break

        print(f"\nAsking Claude about '{topic}'...")

        #--- 4. MAKE THE API REQUEST ---
        # We send the request to the API
        response = client.messages.create(
            model="claude-haiku-4-5-20251001", # We use Haiku because it's fast and cheap
            max_tokens=150, # Limits how long Claude's response can be

            # This is the "Messages Format"
            messages=[
                {
                    "role": "user",
```

```
                    "content": f"Explain the topic '{topic}' in simple terms in under 100 words."
                }
            ]
        )

        # The API returns a complex object. We just want the text content.
        bot_reply = response.content[0].text
        print("\n--- Study Buddy ---")
        print(bot_reply)
        print("--------------------\n")

if __name__ == "__main__":
    main()
```

Execution Output

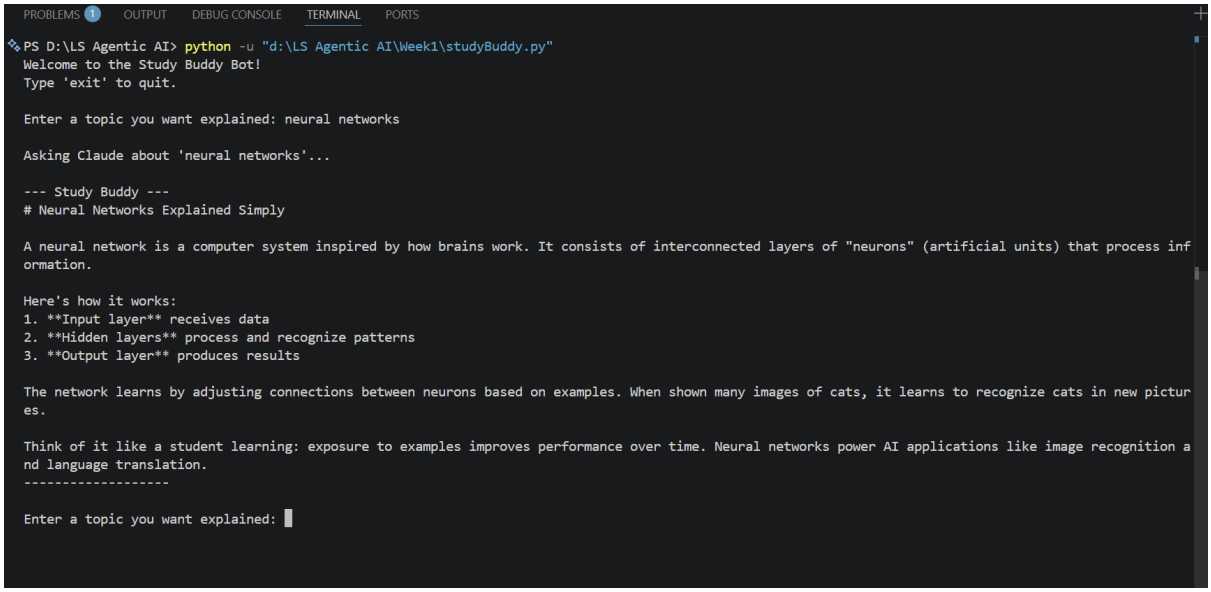

Figure 1: Neural Network output

Question 3: Sentiment Analysis Prompting

1. Zero-Shot Prompt

In zero-shot prompting, the model is given the task with no prior examples to establish a pattern. It relies purely on its pre-trained weights to determine how to respond.

```
Task: Classify the sentiment of the following product review as
    Positive, Negative, or Neutral.

Review: "The delivery was on time but the product stopped
   working after two days. Disappointed."
Sentiment:
```

2. Few-Shot Prompt

In few-shot prompting, the model is provided with a few examples of the expected input-output pairs prior to evaluating the target data. This guides the model's formatting and output structure.

```
Task: Classify the sentiment of the following product reviews
   as Positive, Negative, or Neutral.

Review: "The service was really great. I am extremely delighted
   ."
Sentiment: Positive

Review: "I had quite a bad experience with the service. It was
   awful."
Sentiment: Negative

Review: "The service was fine. It went the normal way as
   expected."
Sentiment: Neutral

Review: "The delivery was on time but the product stopped
   working after two days. Disappointed."
Sentiment:
```

3. Which performs better and why?

The **few-shot prompt** will perform significantly better.

Large Language Models operate as advanced pattern-matching engines. A zero-shot prompt leaves ambiguity, the model might respond with a single word, or it might write a whole conversational paragraph explaining *why* the review is negative. By providing a few-shot prompt, we explicitly establish a strict structural pattern. This constrains the model, drastically reducing hallucination and ensuring it outputs strictly one of the three desired labels without any unwanted conversational filler.

Question 4: Study Schedule Prompt Engineering

---

1. Basic Initial Prompt

A normal student's basic initial prompt might look like this:

"Make a study schedule for me for this week. I need to study for CS 101, MA 110, and PH 107."

2. Two Specific Weaknesses

- **Lack of Realistic Constraints:** The prompt does not specify how many hours the student actually has available per day. Without time bounds, the LLM takes the path of least resistance, it might hallucinate a completely unrealistic 14-hour study day or schedule 8 consecutive hours of a single subject, leading to burnout.

- **Poor Formatting & Unstructured Output:** Without explicit output formatting instructions, the LLM will likely generate a massive, hard-to-read wall of text rather than a visually scannable schedule (like a calendar, list, or table).

3. Improved Prompt

This refined prompt adds constraints, formatting rules, and sets an expected tone:

> Create a weekly study schedule for my three IITB courses: CS 101, MA 110, and PH 107. I have exactly 4 hours available to study on weekdays and 6 hours available on weekends.
>
> **Strict Constraints:**
> - Do not schedule more than 3 hours of any single subject in one day.
> - Include a 15-minute break after every 2 hours of continuous studying.
>
> **Output Format:**
> Present the final schedule as a clean Markdown table with the days of the week as rows, and the allocated subjects/time slots as columns.
>
> **Tone:**
> Keep the text outside of the table brief, encouraging, and highly practical.

4. System Prompt

The system prompt runs in the background to set the AI's persona and core directives:

"You are an expert academic advisor and time-management coach specifically trained to help IIT Bombay students navigate their rigorous coursework. Your objective is to consistently output realistic, highly structured, and burnout-free study schedules based on the exact constraints provided by the user."